

基于深度学习的滑坡识别

一、实验主题：

基于深度学习的滑坡识别（U-Net 语义分割）

二、实验目的：

- 1.学习遥感影像目标识别基础知识，掌握遥感影像语义分割完整流程。
- 2.学习卷积神经网络基本结构，手写实现 U-Net 网络。
- 3.使用开源滑坡数据集 Landslide4Sense 完成模型训练与评估。

三、实验内容：

- （1）数据获取与划分：下载 Landslide4Sense 公开数据集，按固定随机种子（seed=42）将带标签数据划分为训练集与验证集，保留无标签测试集用于预测。
- （2）数据预处理：读取.h5 格式影像，完成通道顺序转换与 Z-score 标准化。
- （3）模型构建：手写实现 U-Net 网络。
- （4）模型训练：配置统一入口用于模型参数设置并完成模型训练。
- （5）模型评估与预测：计算 IoU、F1 Score 等指标，绘制损失曲线、结果标签对比图、无标签数据预测结果。

四、实验过程：

1.研究意义：

传统滑坡研究依赖人工野外调查与专家目视解译，具有效率低、成本高、难以大规模开展的不足，而借助高精度遥感影像与深度学习方法，我们可以迅速、准确地实现大范围滑坡的精确识别，解决传统滑坡调查面临的部分问题。因此，本研究基于 U-Net 语义分割，对 Sentinel-2 多光谱卫星影像进行“滑坡/非滑坡”二分类，实现滑坡范围的像素级刻画。

2.研究区与数据集:

本实验采用 Landslide4Sense (L4S) 公开基准数据集 (Zenodo 记录号 10463239), 其基础数据覆盖日本、印度、尼泊尔、中国台湾这 4 个地理背景差异明显的滑坡多发区。数据集基本属性如表 1 所示:

表 1 L4S 基础信息

属性	说明
数据源	Sentinel-2 多光谱卫星影像, 10m 分辨率
输入通道	12 个光谱波段 (B1~B12)+坡度 (slope)+高程 (elevation)
图像尺寸	128*128
标签	逐像素二值掩膜, 1=滑坡 0=非滑坡

该数据集仅 TrainData 附带标签, TestData 仅含影像、无标签。因此, 本实验将 TrainData (共 3799 个样本) 按固定随机种子切分为带标签的训练集与验证集, 测试集仅用于无标签预测, 具体划分见下方:

表 2 数据集划分

子集	样本数	说明
训练集 train	3554	带标签, 用于模型训练
验证集 val	245	带标签, 提取自 TrainData
测试集 test	800	来自 TestData, 无标签, 仅预测

3.数据预处理:

原始数据使用.h5 格式存储, 每个 image 文件含 img 数据集 (影像数据, 形状为 (128,128,14)), 每个 mask 文件含 mask 数据集 (标签, 形状为 (128,128)), 主要的预处理过程在 dataset/landslide_dataset.py 中的 LandslideDataset 类中完成:

- (1) 读取与维度调整: 读取.h5 格式数据, 将 (H, W, C) 转为 (C, H, W), 以匹配 PyTorch 张量;
- (2) 数据匹配: 匹配 image 影像与 mask 标签;
- (3) 标准化: 逐通道 Z-score 标准化, 对 14 个通道分别执行 $(x - \text{mean}) / \text{std}$, 均值与标准差采用官方在训练集上统计的固定值 (存于 configs/default.yaml), 保证可复现。

4.模型原理:

(1) 卷积神经网络: 卷积神经网络 (CNN) 通过卷积核在特征图上滑动完成局部特征的提取, 配合批归一化 (BatchNorm) 稳定训练、ReLU 引入非线性、池化扩大感受野并降低分辨率, 是语义分割的骨干结构。

(2) U-Net 整体结构: 本实验的基线模型为手写实现的标准 U-Net (参考 github 中 milesial/Pytorch-UNet 仓库), 不依赖 segmentation-models-pytorch 等高层库, 也不加载预训练骨干, 编码器从随机权重开始训练。U-Net 采用“编码器—解码器+跳跃连接”的对称结构: 编码器通过 4 次下采样逐级提取高层语义特征, 解码器通过 4 次上采样逐步恢复空间分辨率; 同时将编码器各阶段的高分辨率特征经跳跃连接 (skip connection) 直接拼接到解码器对应层, 弥补下采样造成的位置信息损失, 从而在保持语义判别能力的同时精细恢复目标边界。上采样采用双线性插值配合卷积 (bilinear=True) 实现, 简单可靠。通道数变化链为: $14 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow \dots \rightarrow 2$ 。

(3) 基础模块: 网络由四个基础模块组装而成 (models/unet/unet_parts.py): DoubleConv 为“ 3×3 卷积 $\rightarrow$ BN $\rightarrow$ ReLU”重复两次的双卷积块; Down 由 2×2 最大池化接 DoubleConv, 实现尺寸减半、通道翻倍; Up 先上采样放大 2 倍, 再与编码器同尺度特征沿通道维拼接后经 DoubleConv 融合; OutConv 用 1×1 卷积将 64 通道压缩为 2 通道。

(4) 输出层: 模型输出 2 个通道 (背景、滑坡各一个 logit), 而非单通道; 推理时对每个像素取 2 通道中概率最大者作为预测类别 ($\text{torch.max}(\text{Pred}, \text{dim}=1)$)。

5.损失函数与优化器:

损失函数采用交叉熵损失 $\text{nn.CrossEntropyLoss}()$: 模型输出 2 通道 logits, 标签为 0/1 类别索引, 损失函数内部完成 Softmax 归一化与类别对齐。该损失由 scripts/losses.py 的 build_loss(cfg) 按配置返回, 当前仅实现 cross_entropy 一种。

优化器采用 Adam, 学习率 $\text{lr}=1 \times 10^{-3}$, 权重衰减 $\text{weight_decay}=5 \times 10^{-4}$ 。实验未设置学习率调度器, 以早停机制充当收敛收尾。

值得说明的是，滑坡像素在数据中占比极低，存在显著的类别不平衡问题，交叉熵对少数类易偏向背景。后续可通过在 `CrossEntropyLoss(weight=...)` 中设置类别权重，或引入 Dice/Focal 损失予以缓解。

6.评估指标:

评估在 `scripts/metrics.py` 的 `compute_metrics` 中完成，基于逐像素二元混淆矩阵（TP/FP/FN/TN）逐类统计，计算指标见下方：

表 3 评估指标

指标	定义	含义
精确率 Precision	$TP/(TP+FP)$	预测为滑坡的像素中真正为滑坡的比例
召回率 Recall	$TP/(TP+FN)$	实际滑坡像素中被成功识别的比例
F1 分数	$2PR/(P+R)$	精确率与召回率的调和平均
IoU（交并比）	$TP/(TP+FP+FN)$	预测与真值滑坡区域的交并比

本报告以滑坡类（第 1 类）的 IoU 与 F1 为主要评价指标，同时报告两类平均的 mean IoU。之所以不采用 Accuracy，是因为滑坡像素占比极低，模型即便将全部像素判为“非滑坡”也能获得较高准确率，无法反映真实的识别能力。

7.实验设置:

实验设置如表 4 所示：

表 4 基础实验设置

项目	取值	说明
模型	U-Net	编码器—解码器+跳跃连接，输出 2 通道
损失函数	CrossEntropyLoss	2 通道输出+类别索引标签
优化器	Adam	$lr=1\times10^{-3}$ ， $weight_decay=5\times10^{-4}$
批大小 batch_size	8	受 4 GB 显存（RTX 3050 Laptop）约束
训练轮数 epochs	30（上限）	实际由早停提前结束
早停 patience	8	监控验证集 landslide_IoU，无提升则停止
随机种子 seed	42	保证每次随机相同
主指标	landslide_IoU/landslide_F1	mean IoU

训练主流程（`train.py`）：每个 epoch 在训练集上执行前向—反向传播更新参数，随后在验证集上计算损失与分割指标；仅当验证集滑坡类 IoU 创新高时保存 `best_model.pt`，连续 8 个 epoch 无提升则触发早停。所有超参数集中在 `configs/default.yaml` 中，修改配置即可调整实验，无需改动主流程代码。

8.实验结果与分析:

训练过程逐轮记录训练损失、验证损失与验证集滑坡类 IoU、F1（完整数据见 runs/exp_01_unet/logs/train_log.csv），结果如表 5 所示：

表 5 逐轮次指标结果

epoch	train_loss	val_loss	landslide_IoU	landslide_F1
0	0.0267	0.0457	0.5250	0.6885
1	0.1025	0.0904	0.3962	0.5676
2	0.1723	0.0437	0.5261	0.6895
3	0.0343	0.0416	0.5259	0.6893
4	0.1799	0.0451	0.5335	0.6958
5	0.0170	0.0443	0.4516	0.6222
6	0.0109	0.0430	0.4515	0.6222
7	0.0274	0.0444	0.4085	0.5801
8	0.0803	0.0402	0.4539	0.6244
9	0.0238	0.0506	0.4742	0.6433
10	0.0477	0.0607	0.4683	0.6379
11	0.0609	0.0423	0.5466	0.7068
12	0.0529	0.0399	0.5584	0.7166
13	0.0198	0.0380	0.5490	0.7088
14	0.0164	0.0363	0.5501	0.7097
15	0.0059	0.0366	0.5290	0.6919
16	0.0066	0.0372	0.5469	0.7071
17	0.0169	0.0353	0.5468	0.7070
18	0.0506	0.0406	0.5689	0.7252
19	0.1127	0.0357	0.5656	0.7226
20	0.0094	0.0380	0.5570	0.7155
21	0.0463	0.0373	0.5256	0.6891
22	0.0337	0.0366	0.5545	0.7134
23	0.0170	0.0395	0.5503	0.7100
24	0.0148	0.0357	0.5561	0.7147
25	0.0317	0.0413	0.4194	0.5910
26	0.0122	0.0384	0.4807	0.6493

注：表中 train_loss 为每个 epoch 最后一个训练批次的损失值（代码以 losses[-1] 记录），故波动较大；val_loss 为验证集整体平均损失，更能反映收敛趋势。

从表中可见，验证集滑坡类 IoU 总体在 0.45~0.57 间波动上升，于第 18 个 epoch 达到最优（IoU=0.5689，F1=0.7252）；此后指标出现波动且不再创新高，最终因连续 8 轮无提升而在第 26 个 epoch 触发早停。训练损失曲线由 train.py 自动保存，如图 1 所示。

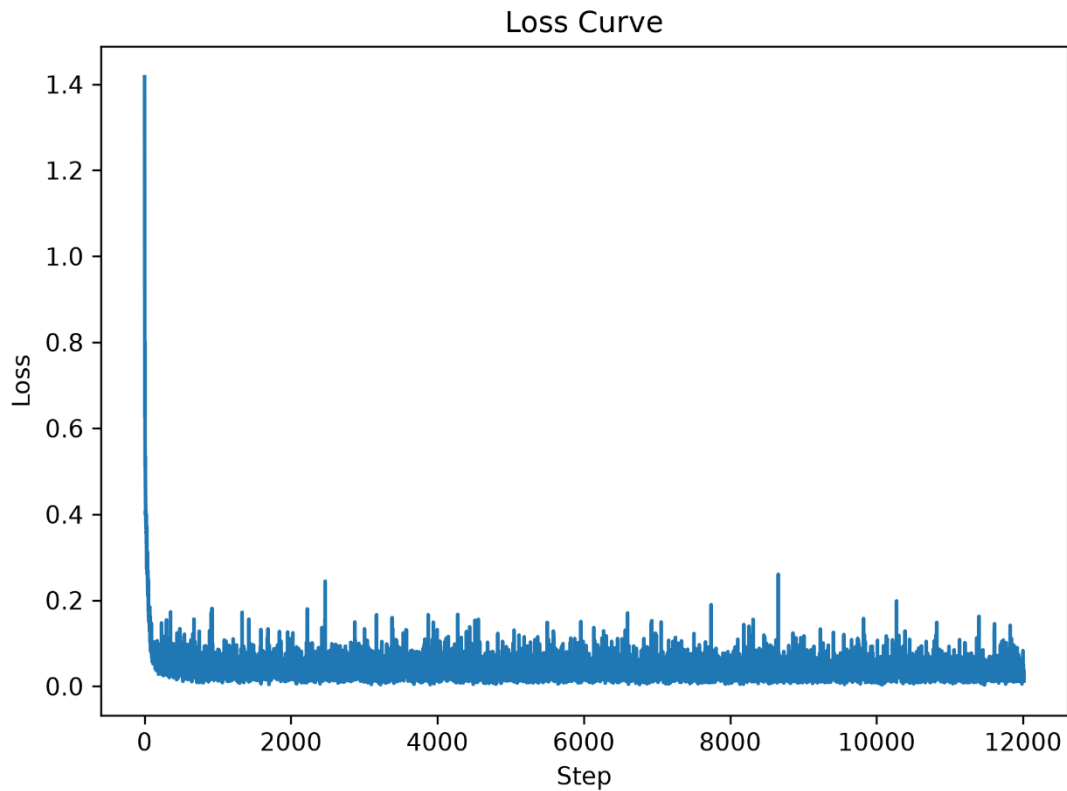


图 1 损失曲线

使用 `evaluate.py` 对模型验证结果进行展示，在验证集上输出真彩色影像、真实掩膜、模型预测对比图（存于 `runs/exp_01_unet/preds/`），图 2 与图 3 为其中两个示例。使用 `predict.py` 对无标签影像进行预测（存于 `runs/exp_01_unet/predictions/`），部分结果如图 4 与图 5 所示。

注意，可视化中，输入影像的“真彩色”显示取 Sentinel-2 第 4/3/2 波段（索引 3/2/1，即红/绿/蓝）组合，以避免波段顺序导致的色彩失真。对比图显示，模型能够较为准确地勾绘主要滑坡体的轮廓，但对小面积、低对比度滑坡存在漏检，对道路、裸露岩体等易产生误检，这与滑坡类别不平衡及样本本身的形态复杂性有关。

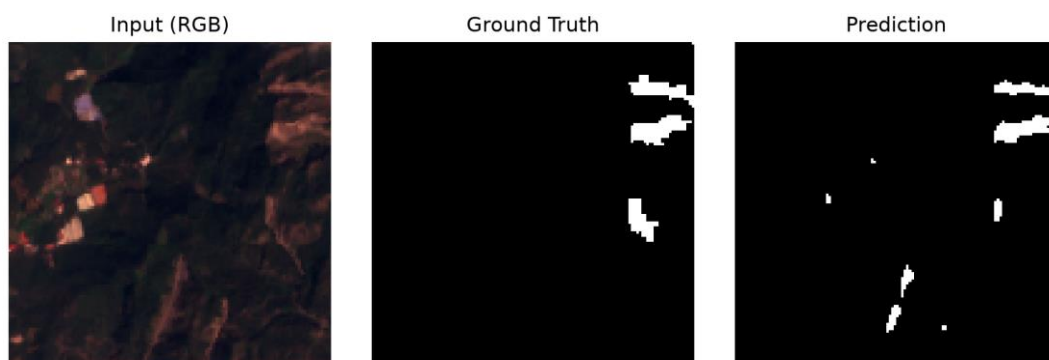


图 2 模型验证结果示例 1

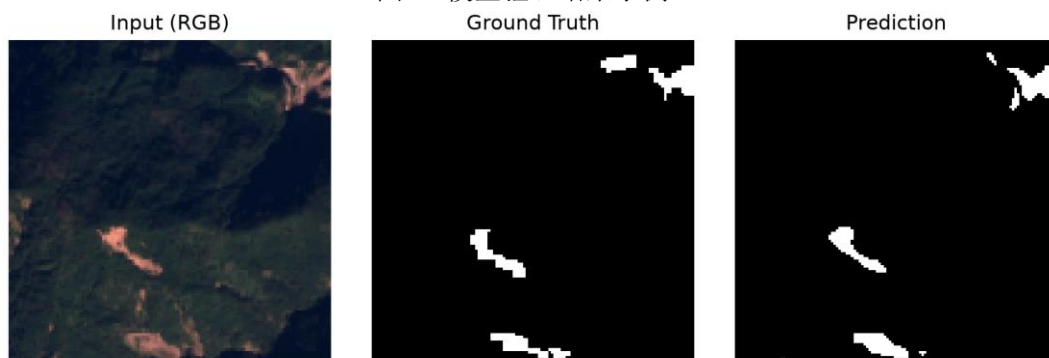


图 3 模型验证结果示例 2

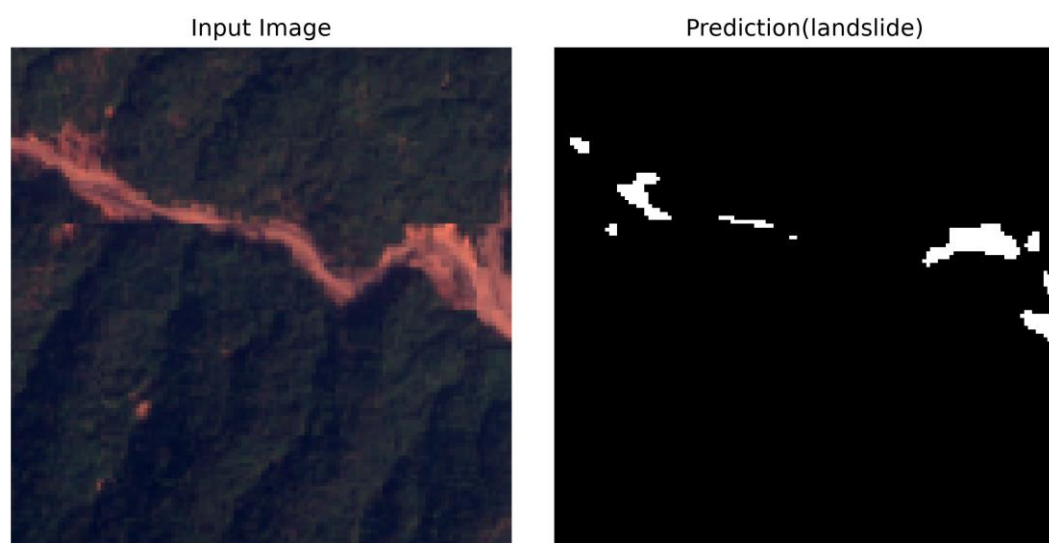


图 4 模型预测示例 1

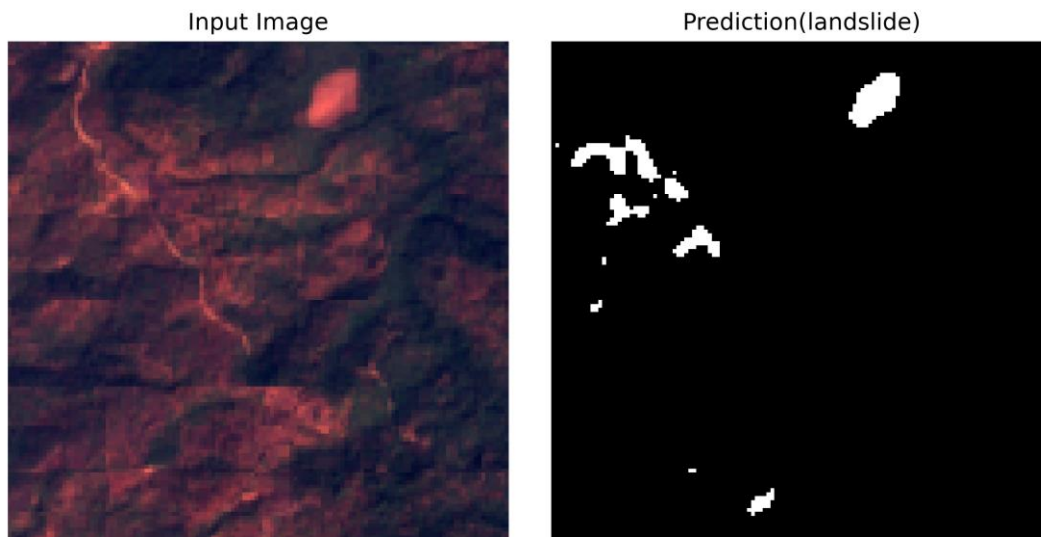


图 5 模型预测结果示例 2

9.项目代码结构：

本项目采用模块化设计，将“数据、模型、损失、指标、训练主流程”分层解耦，并通过统一的配置入口 `configs/default.yaml` 集中管理所有超参数。整体目录结构如下：

```
LandslideDetection/
|
|— README.md                # 项目说明
|— requirements.txt          # 依赖清单
|— configs/
|   |— default.yaml          # 所有超参数集中配置
|— data/                     # 数据目录（大文件，运行时生成）
|   |— raw/
|   |   |— train/  img/  mask/    # 训练集 3554（带标签）
|   |   |— val/   img/  mask/     # 验证集 245（带标签）
|   |   |— test/  img/           # 测试集 800（无标签）
|   |   |— _downloads/          # 下载 zip 临时目录
|   |   |— sample/              # 2 个样例 h5（测试用）
|— dataset/                  # 数据读取模块
|   |— __init__.py
|   |— landslide_dataset.py      # LandslideDataset
|— models/                   # 模型包
|   |— __init__.py              # build_model(cfg) 统一入口
|   |— unet/                    # U-Net 子包
|       |— __init__.py
|       |— unet_model.py         # U-Net 组装
|       |— unet_parts.py         # DoubleConv / Down / Up / OutConv
```



```

├─ train.py                # 训练主循环
├─ evaluate.py            # 加载最优权重在验证集评估+存"输入/真
值/预测"对比图
├─ predict.py            # 对 test 集前 N 张盲预测，存掩膜+概览
PNG
├─ test_unet.py          # 冒烟测试
├─ scripts/              # 工具脚本
│   └─ download_data.py  # 下载 + 切分数据
│   └─ losses.py         # build_loss(cfg)
│   └─ metrics.py        # compute_metrics()
│   └─ test_h5.py        # 打印查看 h5 内部 keys / shape
├─ runs/                 # 实验输出（运行时自动生成）
│   └─ exp_01_unet/
│       ├── checkpoints/best_model.pt # 验证集 landslide_IoU 最优权重
│       └── logs/train_log.csv        # 每轮 train_loss/val_loss/IoU/F1
记录
│       ├── logs/loss.png            # 损失曲线
│       ├── preds/                   # evaluate.py 出的对比图
│       └── predictions/             # predict.py 出的掩膜+概览图
│
└─ docs/                            # 文档 / Docs

```

10.模型拓展：

本次实验仅采用了 U-Net 这一种深度学习模型，但项目文件中可新增自定义模型。主要数据处理与模型训练、评估代码均无需改动。可在 `models/` 下新建模型子包并暴露接口，接着在 `models/__init__.py` 的 `build_model(cfg)` 中新增对应分支即可（仿照 `unet`），最后，修改 `configs/default.yaml` 中的模型名称即可。